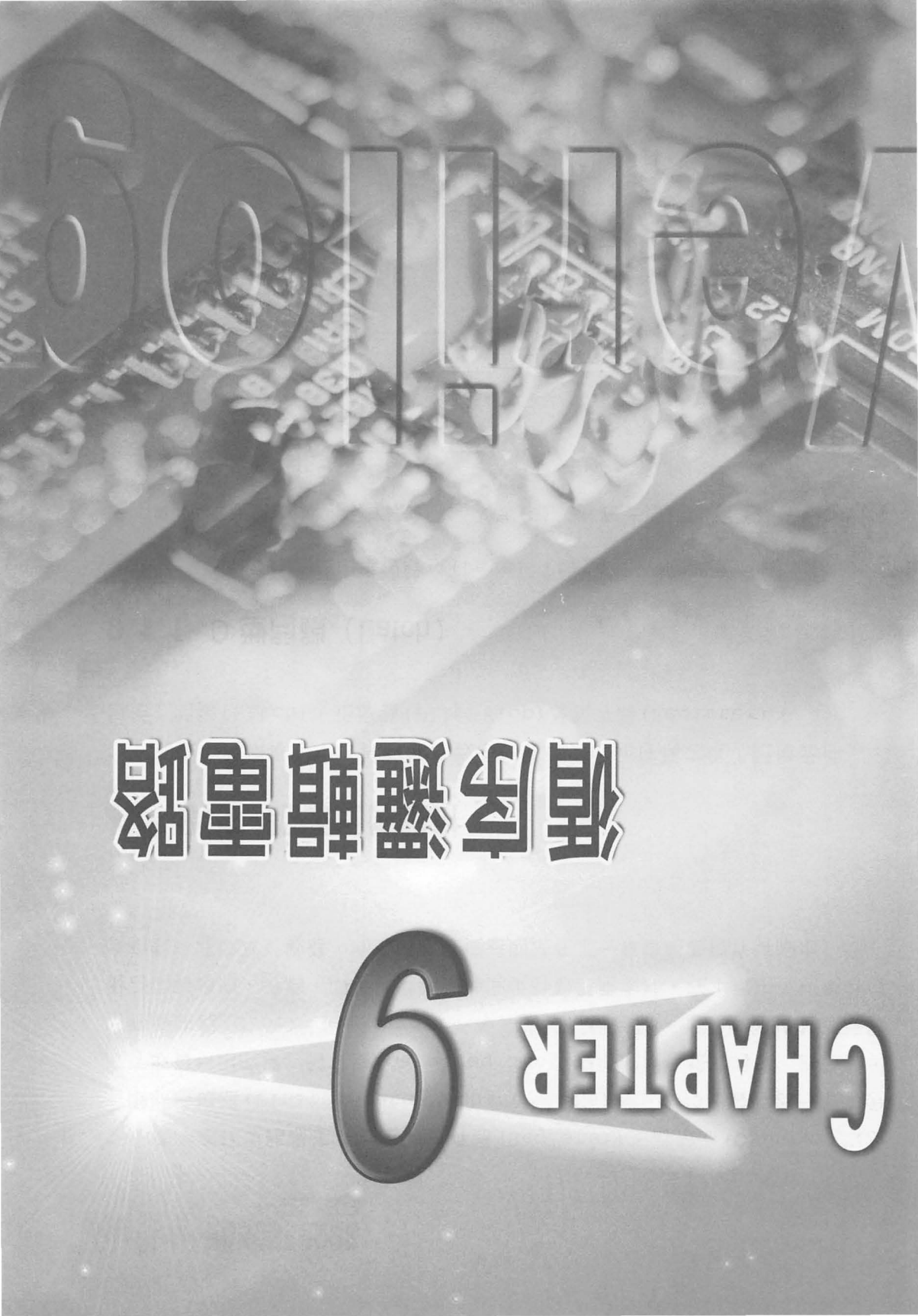




循序邏輯電路

CHAPTER 9

循序邏輯電路最主要包含：暫存器 (Register)、計數器 (Counter) 以及有限狀態機器 (Finite State Machine, FSMs) 等三大類。暫存器一般可分成存取資料用的資料暫存器 (data register) 與用於資料格式轉換、計數器、時序產生器 (Clock Generator) … 等等的移位暫存器。而計數器依其正反器是否同時轉態，分為：同步計數器與非同步計數器兩大類。在本章中，將為各位介紹暫存器以及計數器，而有限狀態機器則是在下一章會為各位作詳細地介紹。



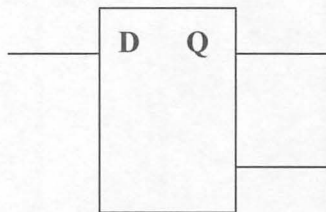
9.1 記憶元件設計

在數位邏輯電路中，具有儲存數位訊號能力的元件稱之為「記憶元件」，包括：閘鎖 (Latch)、正反器 (Flip-Flop) 及暫存器 (Register)。

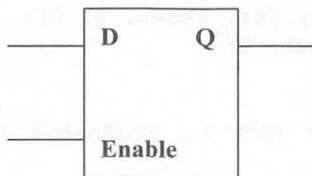
9.1.1 D 型閘鎖 (Latch)

閘鎖 (Latch) 是使用準位觸發 (Level Trigger) 的方式動作的。

- **動作方式**：D 型閘鎖有一個輸入端 (D)，其在一段時間延遲後，輸出將隨著輸入訊號作改變。
- **特性方程式**： $Q = D$



▲ D 型閘鎖的邏輯符號

例 1 具有「輸出致能」訊號控制的準位感測門鎖之邏輯符號

// Latch_D.V: 具有「輸出致能」訊號控制的準位感測門鎖

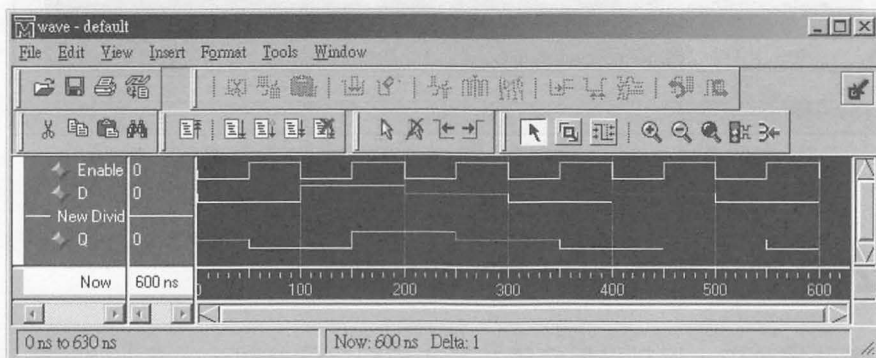
```
module Latch_D (Enable, D, Q);
```

```
input Enable, D;
```

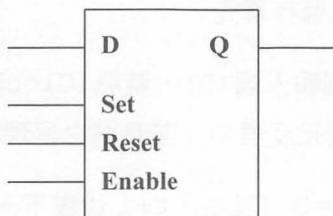
```
output Q;
```

```
assign Q = Enable ? D : Q;
```

```
endmodule
```

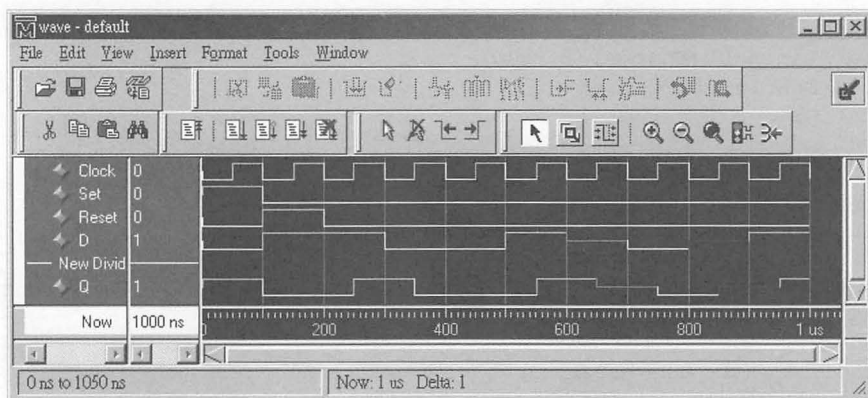


▲ Latch_D.V 的模擬結果

例 2 具有「重置」、「設定」以及「輸出致能」訊號控制的準位感測門鎖之邏輯符號

```
// Latch_RS.V :
// 具有「重置」、「設定」以及「輸出致能」訊號控制的準位感測門鎖
module Latch_RS (Enable, Set, Reset, D, Q);
input  Enable, Set, Reset, D;
output Q;

    assign Q = Reset ? 0 : (Set ? 1 : (Enable ? D : Q) );
endmodule
```

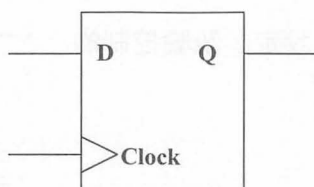


▲ Latch_RS.V 的模擬結果

9.1.2 D 型正反器 (D type Flip-Flop)

在某些場合中，資料只對希望在某些控制訊號啟動時才進入門鎖電路的雙穩態電路稱為正反器。有別於門鎖 (Latch) 的地方在於：正反器 (Flip-Flop) 是使用邊緣觸發 (Edge Trigger) 的方式動作的，而門鎖 (Latch) 是使用準位觸發 (Level Trigger) 的方式動作的。

- D 型正反器為二進位儲存單元。
- D 型正反器具有一個輸入端 (D)、時脈 (Clock Pulse) 訊號，將輸入端的資料取樣後儲存於正反器中，並且將之呈現在輸出端。
- 特性方程式： $Q_{(t+1)} = D$ (其中： $t+1$ 代表下一個時脈來臨的時間)



▲ D 型正反器的邏輯符號

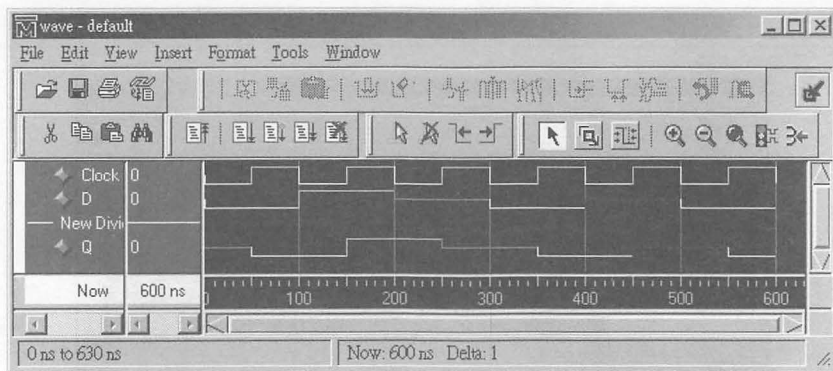
特性表

D	$Q_{(t+1)}$
0	0
1	1

例 1 最簡單的 D 型正反器，直接將 D 傳入 Q

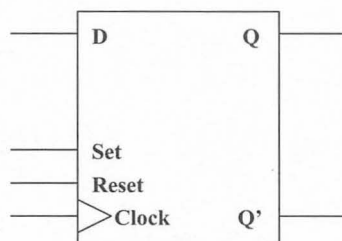
```
// D_FF.V, 最簡單的 D 型正反器，直接將 D 傳入 Q
module D_FF(Clock, D, Q);
input  Clock, D;
output Q;
reg    Q;

    always @(posedge Clock)
    begin
        Q = D; // 直接將 D 傳入 Q
    end
endmodule
```



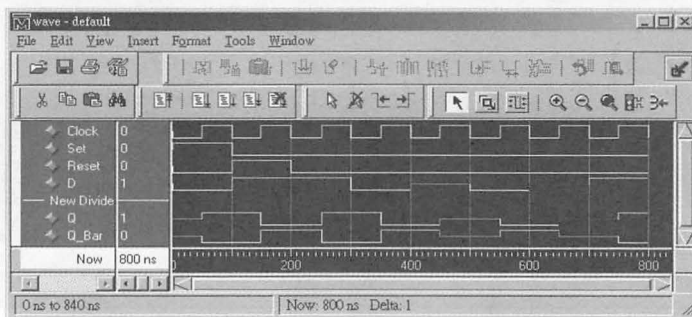
▲ D_FF.V 的模擬結果

例 2 具有「重置」及「設定」訊號控制的正反器的邏輯符號



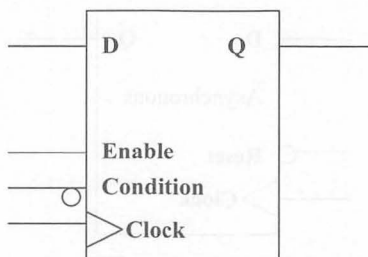
```
// DFF_RS.V: 具有「重置」及「設定」訊號控制的正反器
module DFF_RS (Clock, Set, Reset, D, Q, Q_Bar);
input  Clock, Set, Reset, D;
output Q, Q_Bar;
reg    Q, Q_Bar;

always @(posedge Clock)
begin
    if (Reset) // 重置
    begin
        Q = 0;
        Q_Bar = 1;
    end
    else
    begin
        if (Set) // 設定
        begin
            Q = 1;
            Q_Bar = 0;
        end
        else
        begin // 正常輸入 D 至 Q, Q_Bar = D 取 Not
            Q = D;
            Q_Bar = ~D;
        end
    end
end
endmodule
```



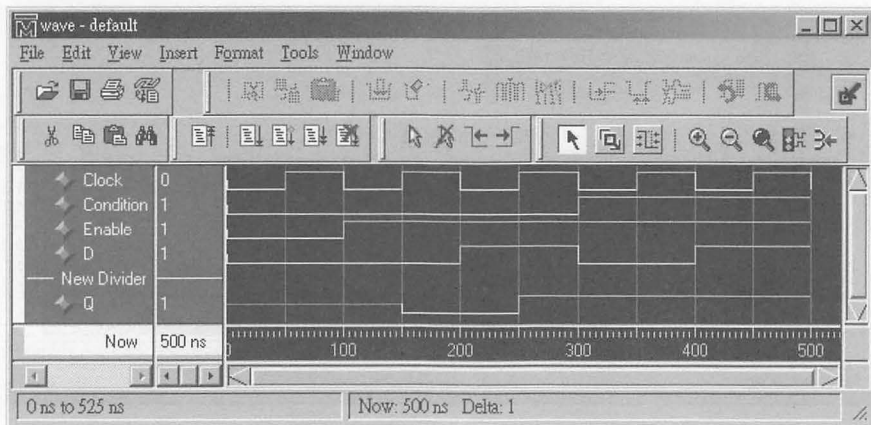
▲ D_RS.V 的模擬結果

例 3 具有「致能」輸出控制訊號的正反器的邏輯符號



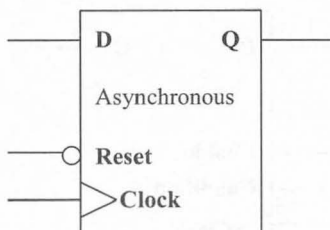
```
// En_FF.V: 具有「致能」輸出控制訊號的正反器
module Enable_FF1 (Clock, Condition, Enable, D, Q);
input  Clock, Condition , Enable, D;
output Q;
reg    Q;

always @(posedge Clock)
begin
    if (Enable) // 「致能」輸出
        Q = (~Condition) ? D : Q; // 若 Condition = 0, 則 Q = D
    else // 否則 Q 值不變
        Q = 1'bx; // 設定為 x (不確定值, Unknown Value)
    end
endmodule
```



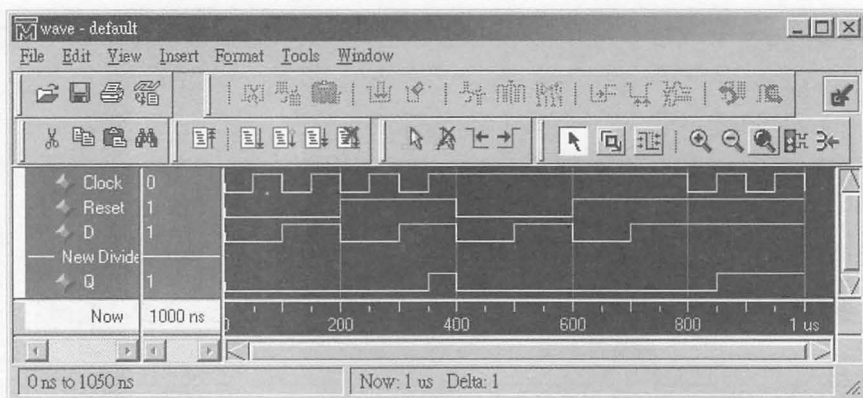
▲ En_FF.V 的模擬結果

例 4 具有非同步重置 (Asynchronous Reset) 的 D 型正反器的邏輯符號



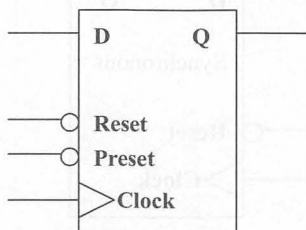
```
// D_FF_AR.V, 具有非同步重置 (Asynchronous Reset) 的 D 型正反器
module D_FF_AR(Clock, Reset, D, Q);
input  Clock, Reset, D;
output Q;
reg    Q;

always @(negedge Reset or posedge Clock)
begin
    if (!Reset) // Reset = 0
        Q = 1'b0; // 清除為 0
    else
        if (Clock) // Reset = 1
            Q = D; // 將 D 傳入 Q
end
endmodule
```

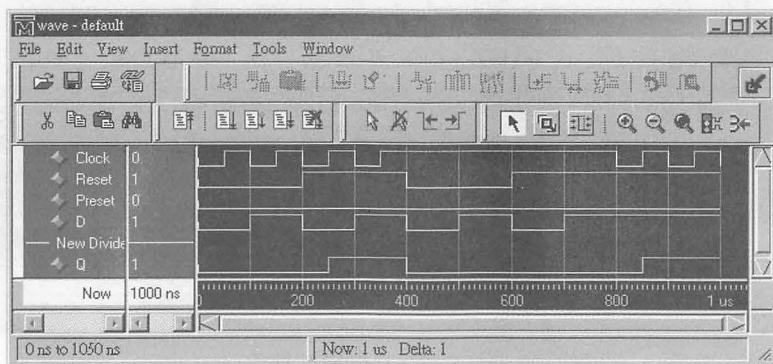


▲ D_FF_AR.V 的模擬結果

例 5 具有非同步 (Asynchronous) 重置 (Reset) 與預設 (Preset) 的 D 型正反器的邏輯符號

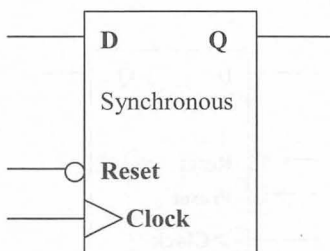


```
// D_FF_PR.V: 具有非同步 (Asynchronous) 重置 (Reset)
// 與預設 (Preset) 的 D 型正反器
module D_FF_PR (Clock, Reset, Preset, D, Q);
input  Clock, Reset, Preset, D;
output Q;
reg    Q;
always @(negedge Reset or negedge Preset or posedge Clock)
begin
    if (!Reset) // Reset = 0
        Q = 1'b0; // 清除為 0
    else
        if (!Preset) // Preset = 0
            Q = 1'b1; // 設定為 1
        else
            if (Clock) // Reset = 1
                Q = D; // 將 D 傳入 Q
end
endmodule
```



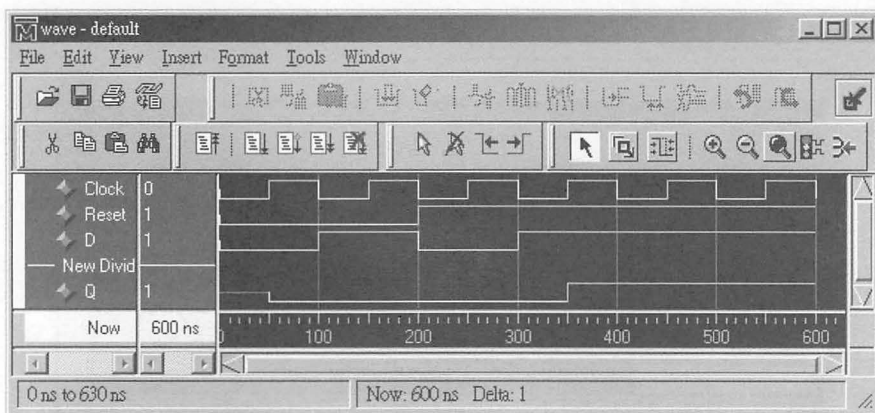
▲ D_FF_PR.V 的模擬結果

例 6 具有同步重置 (Synchronous Reset) 的 D 型正反器的邏輯符號



```
// D_FF_SR.V: 具有同步重置 (Synchronous Reset) 的 D 型正反器
module D_FF_SR (Clock, Reset, D, Q);
input  Clock, Reset, D;
output Q;
reg    Q;

always @(posedge Clock)
begin
    if (!Reset)
        Q = 1'b0; // 清除為 0
    else
        Q = D;    // 將 D 傳入 Q
    end
endmodule
```



▲ D_FF_SR.V 的模擬結果

9.1.3 JK 型正反器 (JK type Flip-Flop)

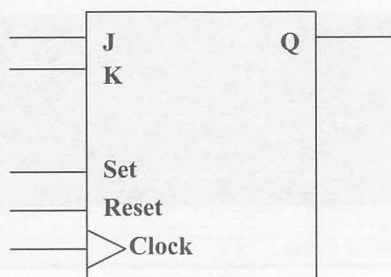
- JK 型正反器也是一種二進位儲存單元。
- 特性方程式： $Q_{(t+1)} = J \cdot /Q_{(t)} + /K \cdot Q_{(t)}$

特性表

J	K	$Q_{(t)}$	$Q_{(t+1)}$
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

($t+1$ 代表下一個時脈來臨的時間)

範例 具有同步「設定」以及「重置」控制訊號的 JK 型正反器



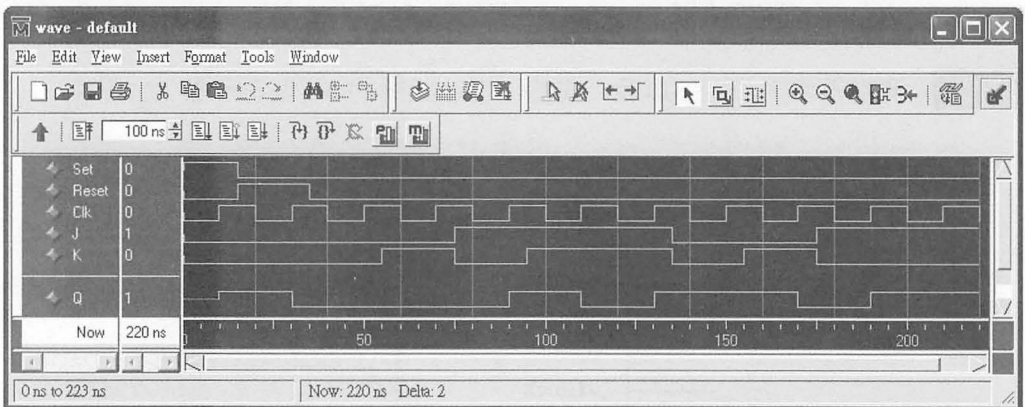
▲ JK_FlipFlop 的邏輯符號

【JK_FlipFlop 的 Verilog 程式碼：JK_FlipFlop.V】

```
// JK_FlipFlop.V, 具有同步「重置」及「設定」訊號控制的 JK 型正反器
module JK_FlipFlop(Set, Reset, Clk, J, K, Q);
input Set, Reset, Clk, J, K;
output Q;
reg Q;

always @(posedge Clk)
begin
    if (Set)
        Q = 1'b1; // 設為 1
    else
        if (Reset)
            Q = 1'b0; // 清除為 0
        else
            if (Clk)
                Q = (J && ~Q) | (~K && Q);
end

endmodule
```



▲ JK_FlipFlop 的模擬結果

9.1.4 T 型正反器 (T type Flip-Flop)

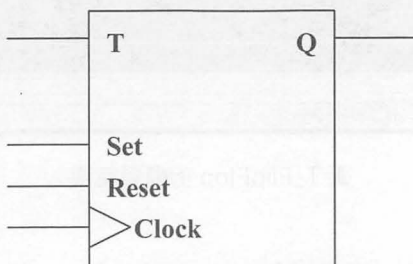
- T 型正反器也是一種二進位儲存單元。
- 特性方程式： $Q_{(t+1)} = \neg T \cdot Q_{(t)} + T \cdot \neg Q_{(t)}$

特性表

T	$Q_{(t)}$	$Q_{(t+1)}$
0	0	0
0	1	1
1	0	1
1	1	0

($t+1$ 代表下一個時脈來臨的時間)

範例 具有同步「設定」以及「重置」控制訊號的 T 型正反器



▲ T_FlipFlop 的邏輯符號

【T_FlipFlop 的 Verilog 程式碼：T_FlipFlop】

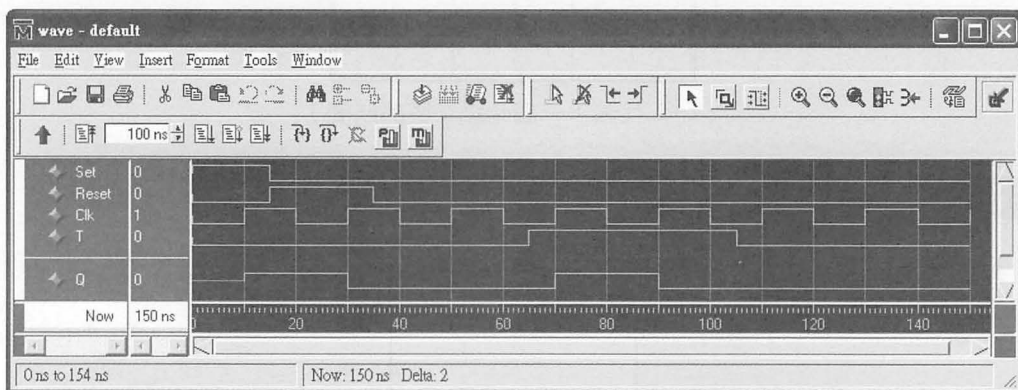
```
// T_FlipFlop.V, 具有同步「重置」及「設定」訊號控制的 T 型正反器
module T_FlipFlop(Set, Reset, Clk, T, Q);
input Set, Reset, Clk, T;
output Q;
reg Q;

always @(posedge Clk)
```

```

begin
  if (Set)
    Q = 1'b1; // 設為 0
  else
    if (Reset)
      Q = 1'b0; // 清除為 0
    else
      if (Clk)
        if (T)
          Q = ~Q;
end
endmodule

```

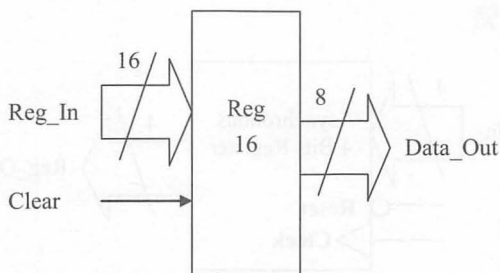


▲ T_FlipFlop 的模擬結果

9.1.5 暫存器 (Register)

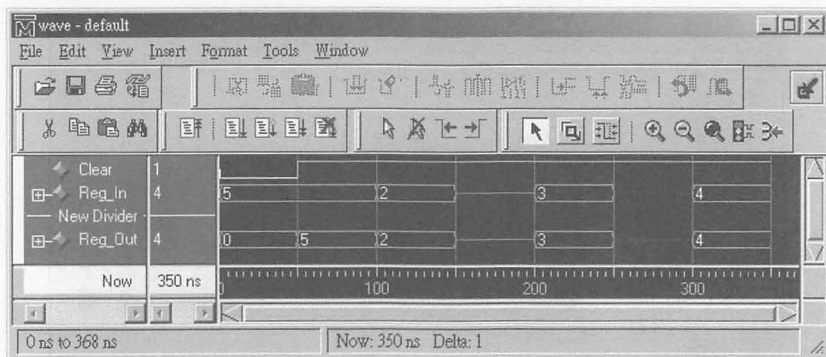
暫存器是一群二進位存取單元的集合，可以用來儲存二進位資料。每一個存取單元通常為一個正反器，因此可以存取一個位元的資料，對於一個具有 n 個位元的暫存器而言，它共可存取 n 個位元的資料。

例 1 具有「清除」訊號控制的 16 位元暫存器的邏輯符號



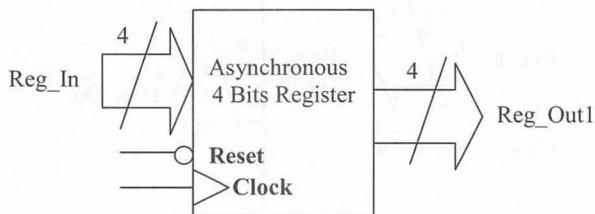
```
// Reg_16.V: 具有「清除」訊號控制的 16 位元暫存器
module Reg_16 (Clear, Reg_In, Reg_Out);
input      Clear;
input      [16:1] Reg_In;
output     [16:1] Reg_Out;
reg        [16:1] Reg_Out;

always @(Clear or Reg_In)
begin
    case (Clear)
        0:
            Reg_Out[16:1] = 16'b0; // 清為 0
        1:
            Reg_Out[16:1] = Reg_In[16:1]; // 載入資料
    endcase
end
endmodule
```



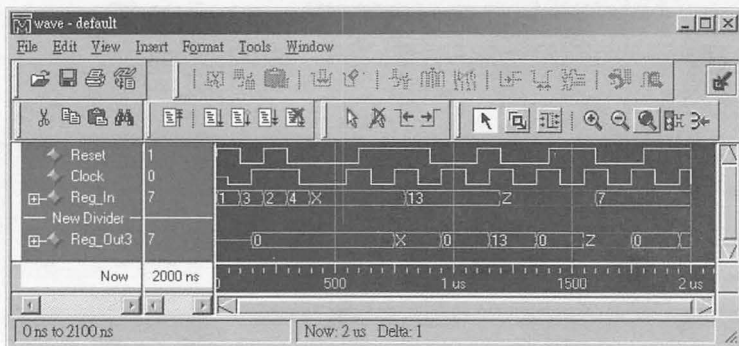
▲ Reg_16.V 的模擬結果

例 2 具有「非同步負緣重置」與「時脈正緣觸發」訊號控制的 4 位元暫存器的邏輯符號



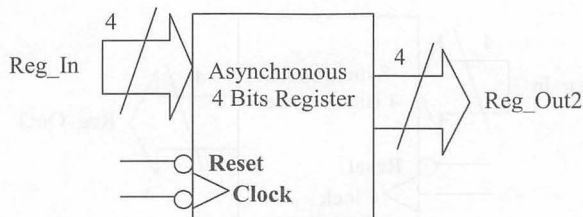
```
// RAS_NRPC.V
// Reg_Out1: 具有「非同步負緣重置」與「時脈正緣觸發」訊號控制的
// 4 位元暫存器
module RAS_NRPC (Reset, Clock, Reg_In, Reg_Out1);
input      Reset, Clock;
input  [3:0] Reg_In;
output  [3:0] Reg_Out1;
reg     [3:0] Reg_Out1;

always @(negedge Reset or posedge Clock)
begin
    if (!Reset) // Reset = 0
        Reg_Out1 = 0; // 清除為 0
    else
        if (Clock) // Reset = 1, Clock = 1
            Reg_Out1 = Reg_In; // 將 Reg_In 傳入 Reg_Out1
end
endmodule
```



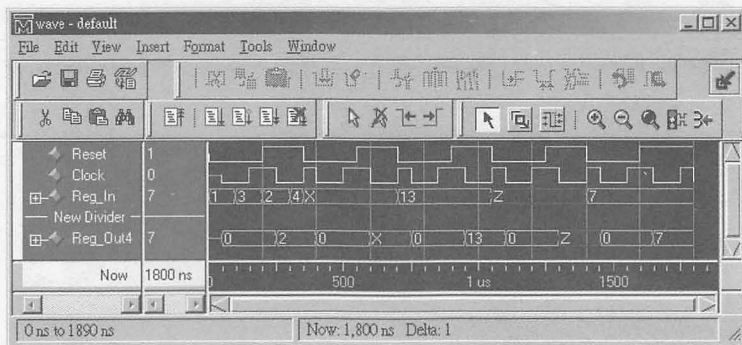
▲ RAS_NRPC.V 的模擬結果

例 3 具有「非同步負緣重置」與「時脈負緣觸發」訊號控制的 4 位元暫存器的邏輯符號



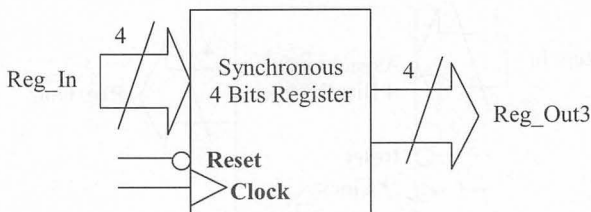
```
// RAS_NRNC.V
// Reg_Out2: 具有「非同步負緣重置」與「時脈負緣觸發」訊號控制的
// 4 位元暫存器
module RAS_NRNC (Reset, Clock, Reg_In, Reg_Out2);
input          Reset, Clock;
input  [3:0]  Reg_In;
output [3:0]  Reg_Out2;
reg  [3:0]  Reg_Out2;

always @(negedge Reset or negedge Clock)
begin
    if (!Reset) // Reset = 0
        Reg_Out2 = 0; // 清除為 0
    else
        if (!Clock) // Reset = 1, Clock = 0
            Reg_Out2 = Reg_In; // 將 Reg_In 傳入 Reg_Out2
        end
end
endmodule
```



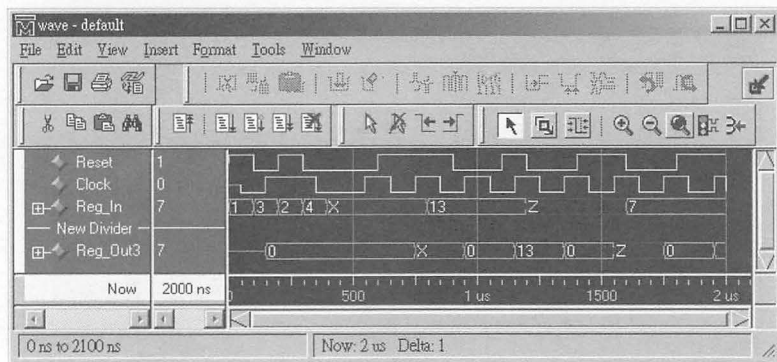
▲ RAS_NRNC.V 的模擬結果

例 4 具有「同步正準位重置」與「時脈正緣觸發」訊號控制的 4 位元暫存器的邏輯符號



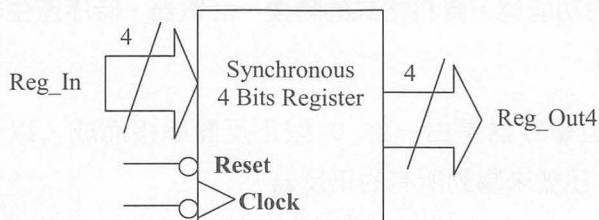
```
// RS_NRPC.V
// Reg_Out3: 具有「同步正準位重置」與「時脈正緣觸發」訊號控制的
// 4 位元暫存器
module RS_NRPC (Reset, Clock, Reg_In, Reg_Out3);
input          Reset, Clock;
input  [3:0]   Reg_In;
output [3:0]   Reg_Out3;
reg   [3:0]   Reg_Out3;

always @(posedge Clock)
begin
    if (!Reset) // Reset = 0
        Reg_Out3 = 0; // 清除為 0
    else
        Reg_Out3 = Reg_In; // 將 Reg_In 傳入 Reg_Out3
end
endmodule
```



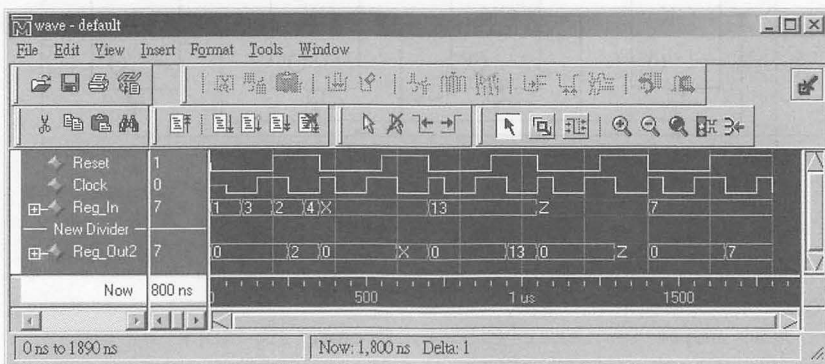
▲ RS_NRPC.V 的模擬結果

例 5 具有「同步負準位重置」與「時脈負緣觸發」訊號控制的 4 位元暫存器的邏輯符號



```
// RS_NRNC.V
// Reg_Out4: 具有「同步負準位重置」與「時脈負緣觸發」訊號控制的
// 4 位元暫存器
module RS_NRNC (Reset, Clock, Reg_In, Reg_Out4);
input      Reset, Clock;
input  [3:0] Reg_In;
output  [3:0] Reg_Out4;
reg     [3:0] Reg_Out4;

always @(negedge Clock)
begin
    if (!Reset) // Reset = 0
        Reg_Out4 = 0; // 清除為 0
    else
        Reg_Out4 = Reg_In; // 將 Reg_In 傳入 Reg_Out4
    end
endmodule
```



▲ RS_NRNC.V 的模擬結果



9.2 移位暫存器 (Shift Register)

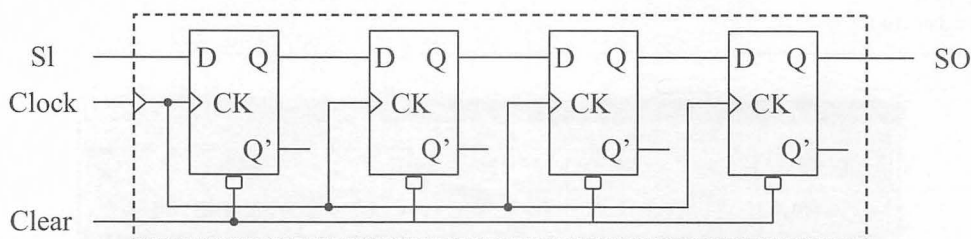
移位暫存器的功能為：資料格式的轉換、計數器、時序產生器 (Clock Generator) … 等等。

最簡單的移位暫存器是由一群 D 型正反器串接而成，以一個共同的時脈 (Clock Pulse) 訊號來驅動所有的正反器。

移位暫存器除了可以儲存資料以外，也可將儲存的資料向左或向右移動。一個具有 n 個位元的移位暫存器通常是由 n 個 D 型正反器連接而成，並以一個共同的時脈 (Clock Pulse) 訊號來驅動。

9.2.1 串進串出 (Serial In Serial Out) 移位暫存器

串進串出 (Serial In Serial Out) 移位暫存器是用一個串列輸入 (Serial Input, SI) 埠用來決定移位期間輸入到最左端那個暫存器的值，串列輸出 (Serial Output, SO) 埠是用來輸出在這次的時脈 (Clock) 訊號來臨之前與上次時脈訊號來臨之間被儲存在最右邊的那個暫存器的值。



例 1 4 位元串進串出、左移、移位暫存器

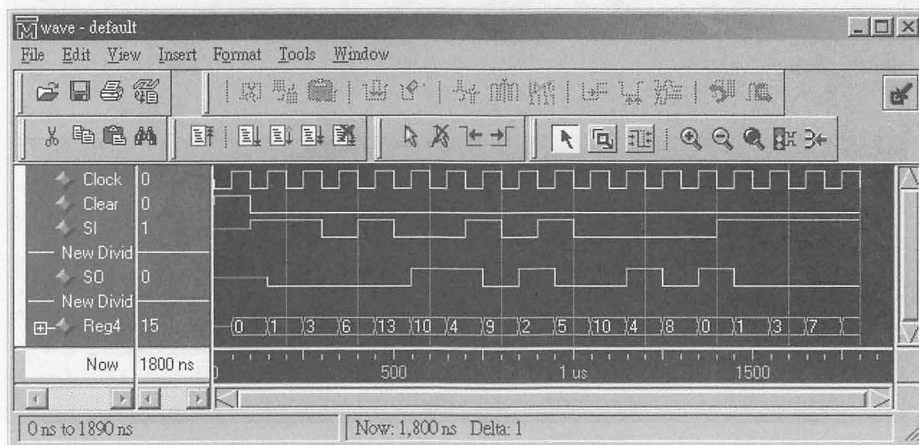
```
// SISO.V: 4 位元串進串出, 左移, 移位暫存器
module SISO_Shift_Register (Clock, Clear, SI, SO);
input    Clock, Clear, SI;
output   SO;
```

```

reg          SO;
reg          [3:0] Reg4;

always @(posedge Clock)
begin
    if (Clear) // 清除
        Reg4 = 4'b0; // 清為 0
    else
    begin
        SO = Reg4[3];
        Reg4[3] = Reg4[2];
        Reg4[2] = Reg4[1];
        Reg4[1] = Reg4[0];
        Reg4[0] = SI; // 移入新的資料 (SI) 至 Reg4[0]
    end
end
endmodule

```



▲ SISO.V 的模擬結果

例 2 4 位元串進串出、左移、移位暫存器 (使用 for 敘述來設計)

```

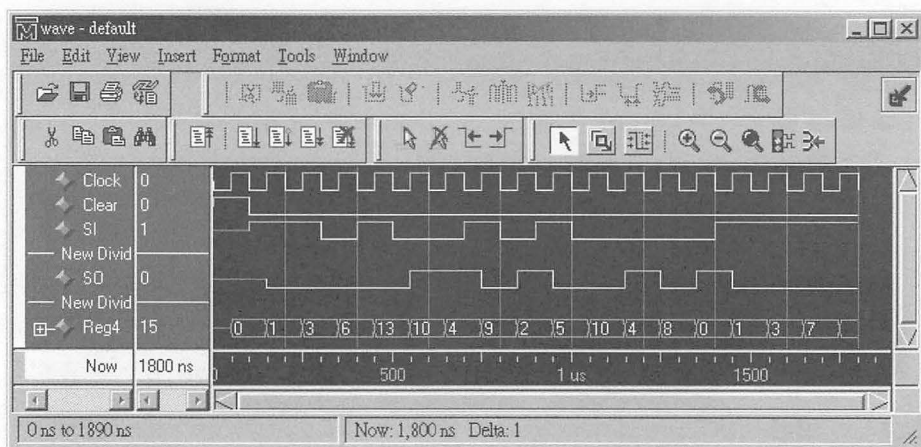
// SISO_for.V: 4 位元串進串出, 左移, 移位暫存器 (使用 for 敘述, 來設計)
module SISO_Shift_Register (Clock, Clear, SI, SO);
input      Clock, Clear, SI;
output     SO;
reg        SO;
reg        [3:0] Reg4;

```

```

always @(posedge Clock)
begin : for_Local // 定義一個區域名稱 for_Local
    integer i; // 定義一個區域性變數
    if (Clear) // 清除
        Reg4 = 4'b0; // 清為 0
    else
    begin
        SO = Reg4[3];
        for (i = 3; i >= 1; i = i - 1)
        begin
            Reg4[i] = Reg4[i-1];
        end
        Reg4[0] = SI; // 移入新的資料 (SI) 至 Reg4[0]
    end
end
endmodule

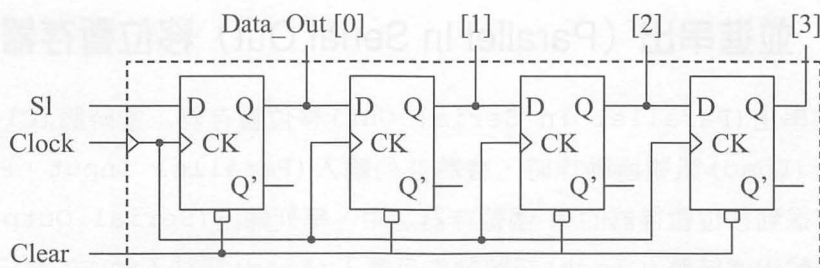
```



▲ SISO_for.V 的模擬結果

9.2.2 串進並出 (Serial In Parallel Out) 移位暫存器

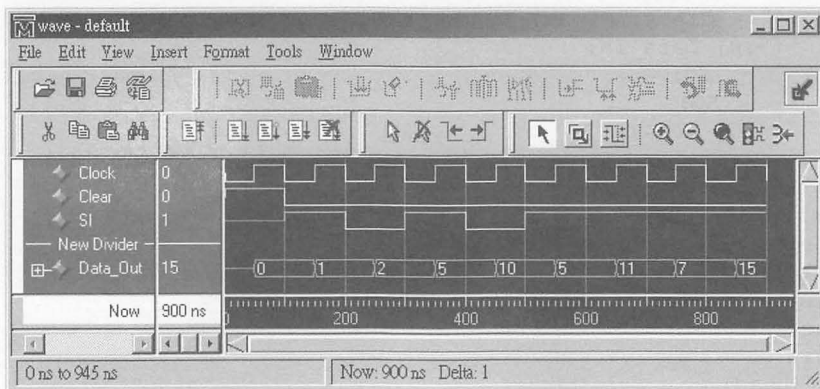
串進並出 (Serial In Parallel Out) 移位暫存器是用一個串列輸入 (Serial Input, SI) 埠用來決定移位期間輸入到最左端那個暫存器的值，並列輸出 (Parallel Output, PO) 埠是用來輸出在這次的時脈 (Clock) 訊號動作之後儲存在 n 個位元移位暫存器內的值。



範例 4 位元串進並出、左移、移位暫存器

```
// SIPO.V: 4 位元串進並出, 左移, 移位暫存器
module SIPO_Shift_Register (Clock, Clear, SI, Data_Out);
input  Clock, Clear, SI;
output [3:0] Data_Out;
reg    [3:0] Data_Out;

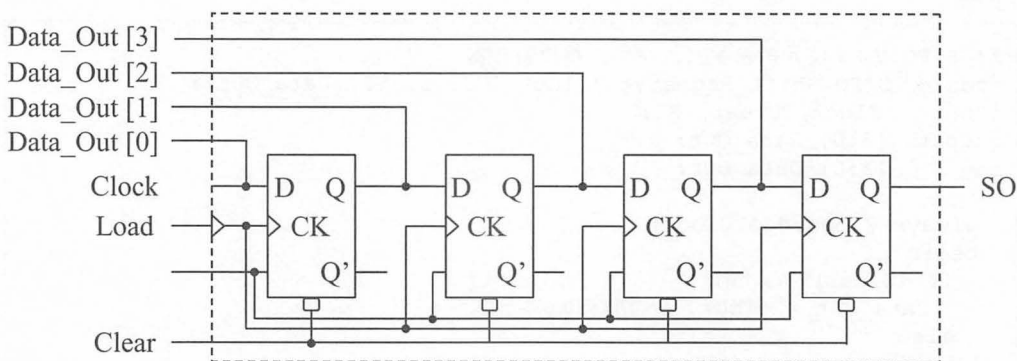
always @(posedge Clock)
begin
    if (Clear) // 清除
        Data_Out = 4'b0; // 清為 0
    else
        begin
            Data_Out = Data_Out << 1; // 左移 1 位元
            Data_Out[0] = SI; // 移入新的資料 (SI) 至 Data_Out[0]
        end
end
endmodule
```



▲ SIPO.V: 4 位元串進並出、左移、移位暫存器合成後，電路的模擬結果

9.2.3 並進串出 (Parallel In Serial Out) 移位暫存器

並進串出 (Parallel In Serial Out) 移位暫存器：當時脈 (Clock) 訊號及載入 (Load) 訊號皆動作時，會將並列輸入 (Parallel Input, PI) 埠上的資料傳送到移位暫存器的 n 個暫存器之中，串列輸出 (Serial Output, SO) 埠是用來輸出當時脈 (Clock) 訊號動作且載入 (Load) 訊號不動作的情況下，儲存在最右邊的那個暫存器的值。



範例 4 位元並進串出、左移、移位暫存器

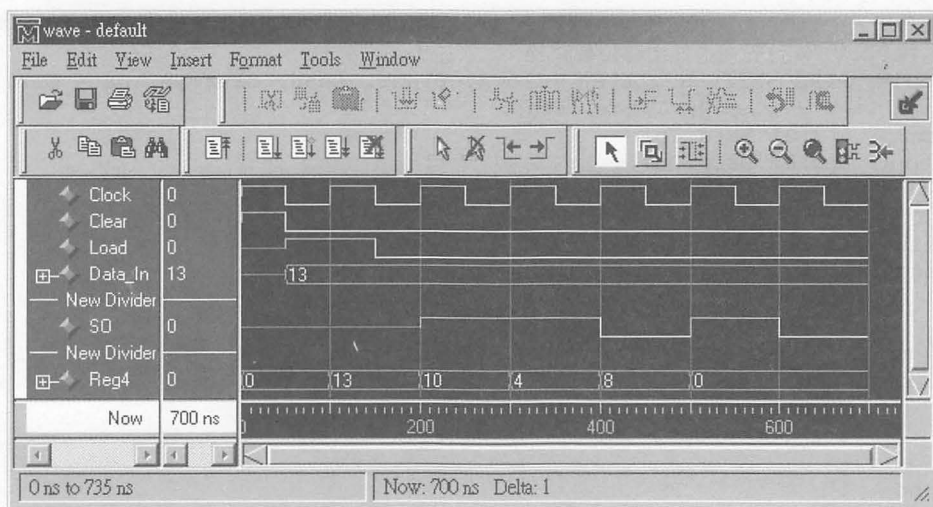
```
// PISO.V: 4 位元並進串出, 左移, 移位暫存器
module PISO_Shift_Register (Clock, Clear, Load, Data_In, SO);
input      Clock, Clear, Load;
input      [3:0] Data_In;
output     SO;
reg        SO;
reg        [3:0] Reg4;

always @(posedge Clock)
begin : for_Local // 定義一個區域名稱 for_Local
    integer i; // 定義一個區域性變數
    if (Clear) // 清除
        Reg4 = 4'b0; // 清為 0
    else
    begin
        if (Load)
            Reg4 = Data_In; // 從 Data_In 載入資料
    end
end
```

```

else
begin
    SO = Reg4[3];
    for (i = 3; i >= 1; i = i - 1)
    begin
        Reg4[i] = Reg4[i-1];
    end
    Reg4[0] = 0; // Reg4[0] 填入 0
end
end
end
endmodule

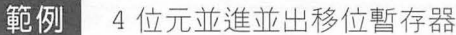
```



▲ PISO.V：4 位元並進串出、左移、移位暫存器合成後，電路的模擬結果

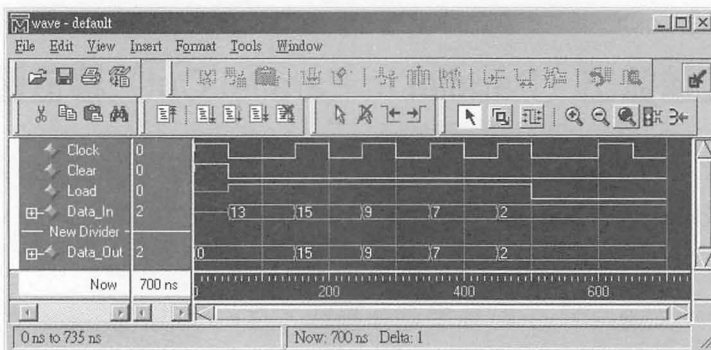
9.2.4 並進並出（Parallel In Parallel Out）移位暫存器

並進並出 (Parallel In Parallel Out) 暫存器：當時脈 (Clock) 訊號及載入 (Load) 訊號皆動作時，會將並列輸入 (Parallel Input, PI) 埠上的資料傳送到移位暫存器的 n 個暫存器之中，並列輸出 (Parallel Output, PO) 埠是用來輸出當時脈 (Clock) 訊號動作且載入 (Load) 訊號不動作的情況下，儲存在 n 個位元移位暫存器內的值。



```
// PIP0.V: 4 位元並進並出移位暫存器
module PIP0_Shift_Register (Clock, Clear, Load, Data_In, Data_Out);
input      Clock, Clear, Load;
input      [3:0] Data_In;
output     [3:0] Data_Out;
reg        [3:0] Data_Out;

always @(posedge Clock)
begin
    if (Clear) // 清除
        Data_Out = 4'b0; // 清為 0
    else
        begin
            if (Load)
                Data_Out = Data_In; // 從 Data_In 載入資料
        end
    end
endmodule
```



▲ PIPO.V：4 位元並進串出移位暫存器合成後，電路的模擬結果



9.3 計數器電路 (Counter)

一般的計數器電路是由一些正反器與一些控制正反器狀態改變用的組合邏輯電路所組成。

計數器依其正反器是否同時轉態，分爲：「同步計數器」與「非同步計數器」兩大類。

- 在同步計數器中，所有正反器均被一個電路中共同的時脈 (Clock) 在同一時間改變狀態。
- 在非同步計數器中，後級的正反器狀態的改變是由前級正反器的輸出所觸發的，所以常被稱爲「漣波計數器」(Ripple Counter)。

計數器依其計數的順序，分爲：「上數計數器」(Count Up Counter)、「下數計數器」(Count Down Counter) 及「上數/下數計數器」(Up/Down Counter)。

對於一個除以 N (稱爲：模 N ，module N) 的計數器而言，依其計數的最大值是否爲 2^n ，可分爲：「模 $N (=2^n)$ 計數器」與「模 $N (\neq 2^n)$ 計數器」。

- 模 $N (=2^n)$ 計數器

對一個「模 8 上數計數器 ($N=8$ 、 $n=3$ ，3 個位元上數計數器)」而言，它的計數範圍由 0、1、2... 到 6、7，最後回到 0、1、2... 等循環計數的方式。因爲計數的最大範圍 $=2^n$ ，而稱之。

- 模 $N (\neq 2^n)$ 計數器

對一個「模 5 上數計數器 ($N=5$)」而言，它的計數範圍由 0、1、2... 到 4、5，最後回到 0、1、2... 等循環計數的方式。因爲計數的最大範圍 $\neq 2^n$ ，稱之。

9.3.1 上數計數器 (Count Up Counter)

「上數計數器」又稱為「正數計數器」。在非同步的上數計數器的內部中，其後級的時脈訊號是由前一級的非補數輸出端取得的。

以「3 個位元的上數計數器」而言，它的計數範圍由 0、1、2... 到 6、7，最後回到 0、1、2... 等循環計數的方式。

9.3.2 下數計數器 (Count Down Counter)

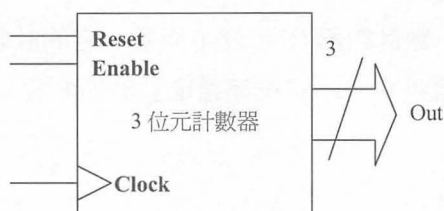
「下數計數器」又稱為「倒數計數器」。在非同步的上數計數器的內部中，其後級的時脈訊號是由前一級的補數輸出端取得的。

以 3 個位元的下數計數器而言，它的計數範圍由 7、6、5... 到 1、0，最後回到 7、6、5... 等循環計數的方式。

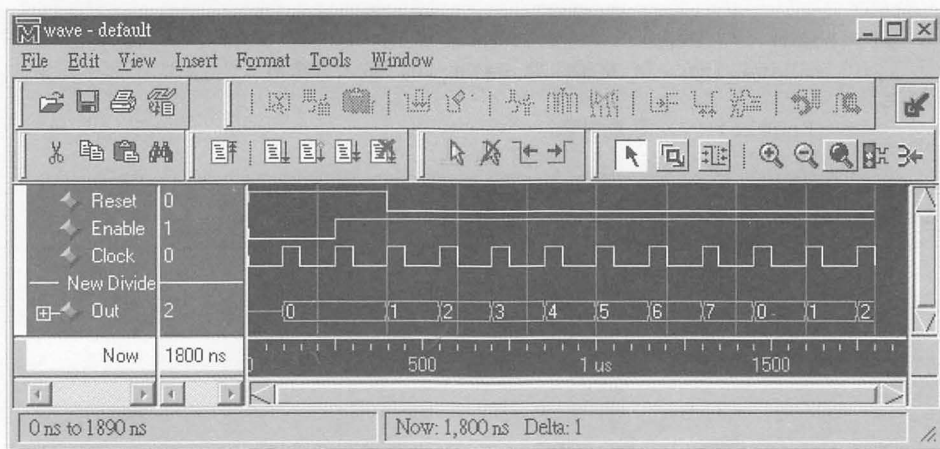
9.3.3 上數／下數計數器 (Up／Down Counter)

「上數／下數計數器」又稱為「正／倒數計數器」，它具有上數及下數的功能，通常用一個「上／下數」(Up／Down) 的控制訊號來決定電路是要作上數或下數的計數。在一個上數／下數計數器的內部中，可以用一個 2 對 1 的多工器選擇正反器後級的時脈訊號是由前一級的非補數輸出端取得的來形成上數的功能，由前一級的補數輸出端取得的來形成下數的功能。

例 1 具有「重置」及「致能輸出」訊號控制的 3 位元 (0~7) 正緣觸發上數計數器

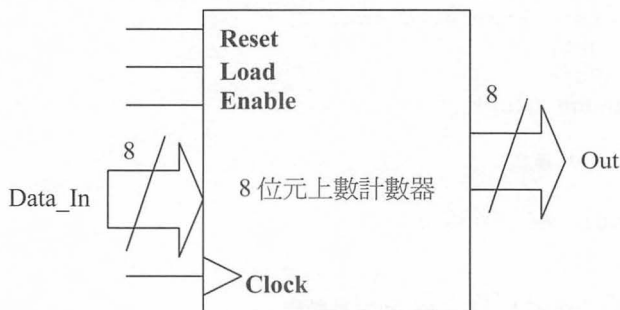


```
// Counter1.V: 計數器, 範例 1
// 具有「重置」及「致能」訊號控制的
// 3 位元 (0~7) 正緣觸發計數器
module Counter1 (Reset, Enable, Clock, Out);
input      Reset, Enable, Clock;
output [2:0] Out;
reg [2:0] Out;
always @(posedge Clock)
begin
    if (Reset) // 重置
    begin
        Out = 3'b0; // 清除為 0
    end
    else
    begin
        if (Enable == 1'b1) // 致能計數器
        begin
            if (Out == 3'd7) // 已經數到最大值了
            begin
                Out = 3'b0; // 清除為 0
            end
            else
            begin
                Out = Out + 1'b1; // 計數值 +1
            end
        end
    end
end
endmodule
```

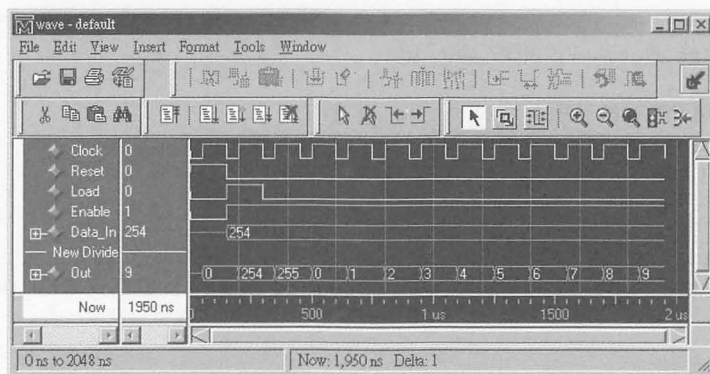


▲ Counter1.V 的模擬結果

例 2 具有「重置」、「載入」及「致能輸出」訊號控制的 8 位元 (0~255) 正緣觸發上數計數器

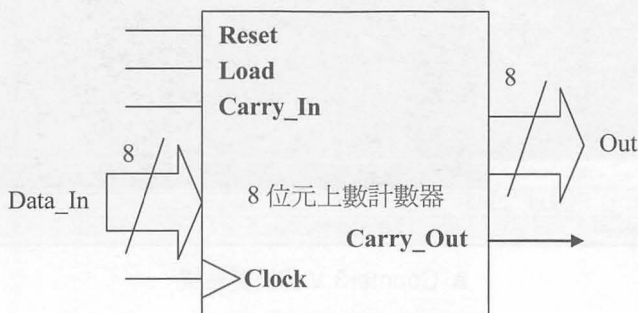


```
// Counter2.V: 上數計數器, 範例 2
// 具有「重置」、「載入」及「致能」訊號控制的
// 8 位元 (0~255) 正緣觸發上數計數器
module Counter2 (Clock, Reset, Load, Enable, Data_In, Out);
input          Clock, Reset, Load, Enable;
input  [7:0]   Data_In;
output  [7:0]  Out;
reg  [7:0]    Out;
always @ (posedge Clock)
begin
    if (Reset) // 重置
        Out = 0; // 清除為 0
    else
        if (Load) // 載入
            Out = Data_In; // 從輸入埠 Data_In, 載入計數值
        else
            if (Enable) // 致能計數
                Out = Out + 1; // 計數值 +1
end
endmodule
```



▲ Counter2.V 的模擬結果

例 3 具有「重置」及「載入」訊號控制的 8 位元 (0~255) 正緣觸發上數計數器



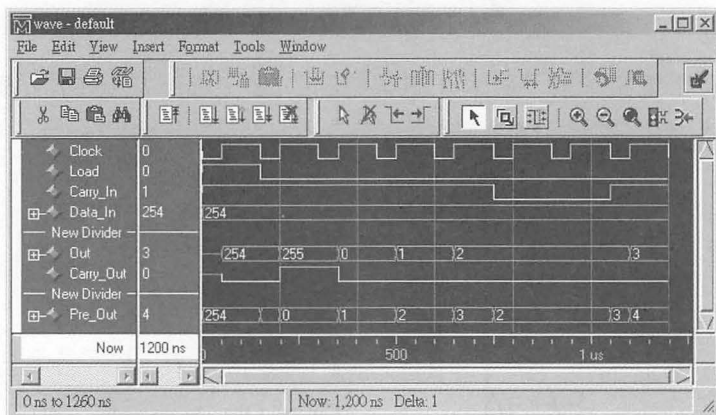
```
// Counter3.V: 8 位元 (0~255) 正緣觸發上數計數器
// 具有「重置」及「載入」訊號控制的
// 8 位元 (0~255) 正緣觸發上數計數器
module Counter3 (Clock, Load, Carry_In, Data_In, Out, Carry_Out);
input      Clock, Load, Carry_In;
input  [7:0] Data_In;
output  [7:0] Out;
output      Carry_Out;
reg      [7:0] Out;
reg      Carry_Out;
reg      [7:0] Pre_Out;

// 產生一個 8 位元的暫存器
always @(posedge Clock) // 當 Clock 正緣觸發時
```

```

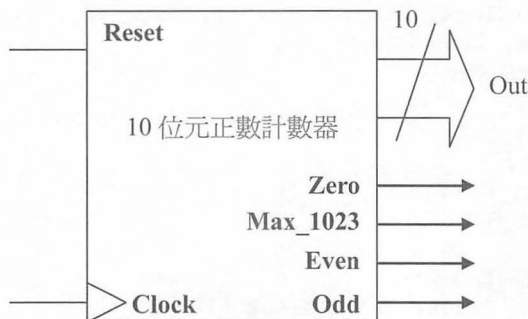
begin
    Out = Pre_Out; // 更新 Out 值
end
// 計算下一個計數器的狀態及 Carry_Out 的值
// 基於「效能」上的考量，我們不希望 Load 訊號影響 Carry_Out
always @(Load or Data_In or Carry_In or Out)
begin
    {Carry_Out, Pre_Out} = Out + Carry_In;
    if (Load) // 載入
        Pre_Out = Data_In; // 從輸入埠 Data_In，載入計數值到 Pre_Out
    end
endmodule

```



▲ Counter3.V 的模擬結果

例 4 10 位元 (0~1023) 正緣觸發上數計數器，將判斷計數值的寫法，置於 always 區塊中，因此「效能」(performance)較差。

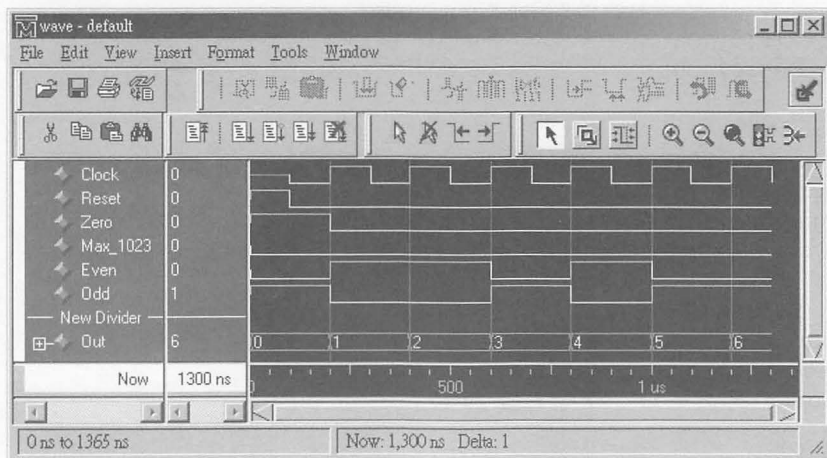


```

// Counter4.V: 10 位元 (0~1023) 正緣觸發上數計數器, 寫法 1
// 具有「重置」訊號控制的 10 位元 (0~1023) 正緣觸發上數計數器
// 若計數值為 0, 則 Zero = 1
// 若計數值為 1023, 則 Max_1023 = 1
// 若計數值為偶同位, 則 Even = 1
// 若計數值為偶同位, 則 Odd = 1
// 將判斷計數值的寫法, 置於 always 區塊中「效能」較差
module Counter4 (Clock, Reset, Zero, Max_1023, Even, Odd, Out);
input      Clock, Reset;
output     Zero, Max_1023, Even, Odd;
reg        Zero, Max_1023, Even, Odd;
output     [9:0] Out;
reg        [9:0] Out;

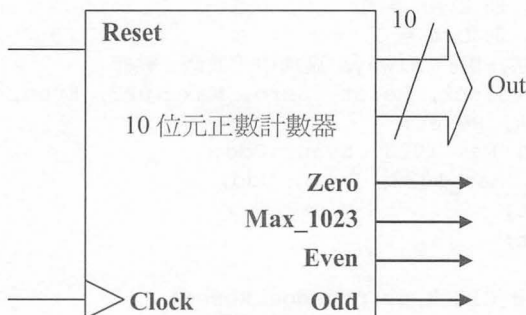
always @(posedge Clock or posedge Reset)
begin
    if (Reset) // 重置
        Out = 0; // 清除為 0
    else
        Out = Out + 1; // 計數值 加 1
    Zero = ! (| Out);
    Max_1023 = & Out;
    Even = ^ Out;
    Odd = ! (Even);
end
endmodule

```



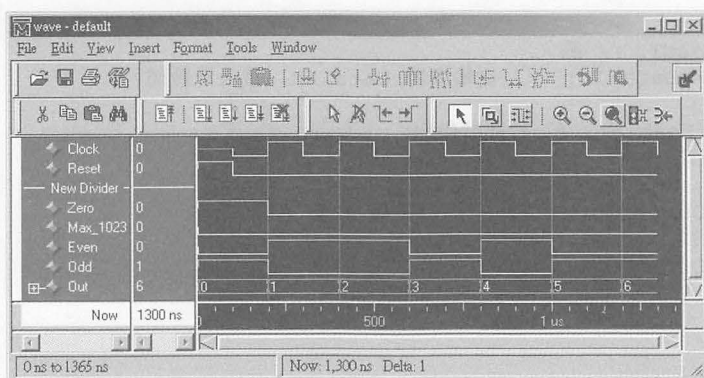
▲ Counter4.V 的模擬結果

例 5 10 位元 (0~1023) 正緣觸發上數計數器，將判斷計數值的寫法，置於 always 區塊外，因此「效能」(performance)較佳。



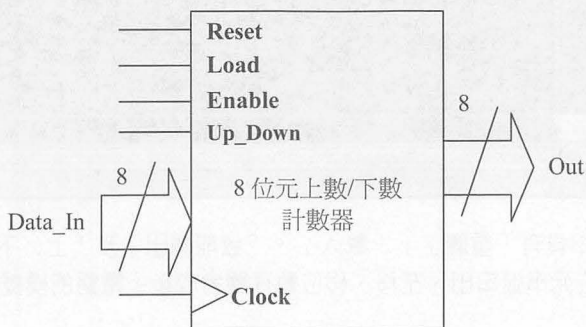
```
// Counter5.V: 10 位元 (0~1023) 正緣觸發上數計數器，寫法 2
// 具有「重置」訊號控制的 10 位元 (0~1023) 正緣觸發上數計數器
// 若計數值為 0，則 Zero = 1
// 若計數值為 1023，則 Max_1023 = 1
// 若計數值為偶同位，則 Even = 1
// 若計數值為偶同位，則 Odd = 1
// 將判斷計數值的寫法，置於 always 區塊外「效能」較佳
module Counter5 (Clock, Reset, Zero, Max_1023, Even, Odd, Out);
input      Clock, Reset;
output     Zero, Max_1023, Even, Odd;
wire      Zero, Max_1023, Even, Odd;
output     [9:0] Out;
reg        [9:0] Out;

assign Zero = ! (| Out);
assign Max_1023 = & Out;
assign Even = ^ Out;
assign Odd = ! (Even);
always @(posedge Clock or posedge Reset)
begin
    if (Reset) // 重置
        Out = 0; // 清除為 0
    else
        Out = Out + 1; // 計數值 加 1
    end
endmodule
```



▲ Counter5.V 的模擬結果

例 6 具有「重置」、「載入」、「致能」及「上／下數」訊號控制的 8 位元 (0~255) 正緣觸發上數／下數計數器



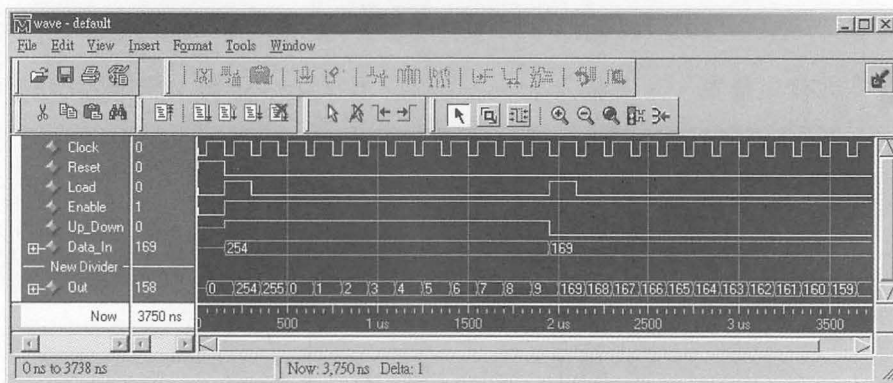
```
// Counter6.V: 上數計數器, 範例 6
// 具有「重置」、「載入」、「致能」及「上/下數」訊號控制的
// 8 位元 (0~255) 正緣觸發上數/下數計數器
module Counter6 (Clock, Reset, Load, Enable, Up_Down, Data_In,
Out);
input      Clock, Reset, Load, Enable, Up_Down;
input [7:0] Data_In;
output [7:0] Out;
reg [7:0] Out;

always @ (posedge Clock)
begin
    if (Reset) // 重置
        Out = 0; // 清除為 0
end
```

```

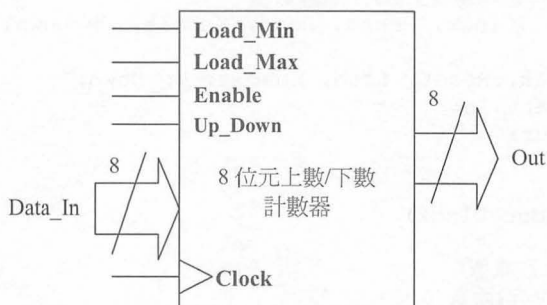
else
  if (Load) // 載入
    Out = Data_In; // 從輸入埠 Data_In, 載入計數值
  else
    if (Enable) // 致能計數
      begin
        if (Up_Down) // 上數/下數
          Out = Out + 1; // 計數值 +1, 上數
        else
          Out = Out - 1; // 計數值 -1, 下數
        end
      end
    end
  end
endmodule

```



▲ Counter6.V：具有「重置」、「載入」、「致能輸出」及「上／下數」訊號控制的 4 位元串進串出、左移、移位暫存器合成後，電路的模擬結果

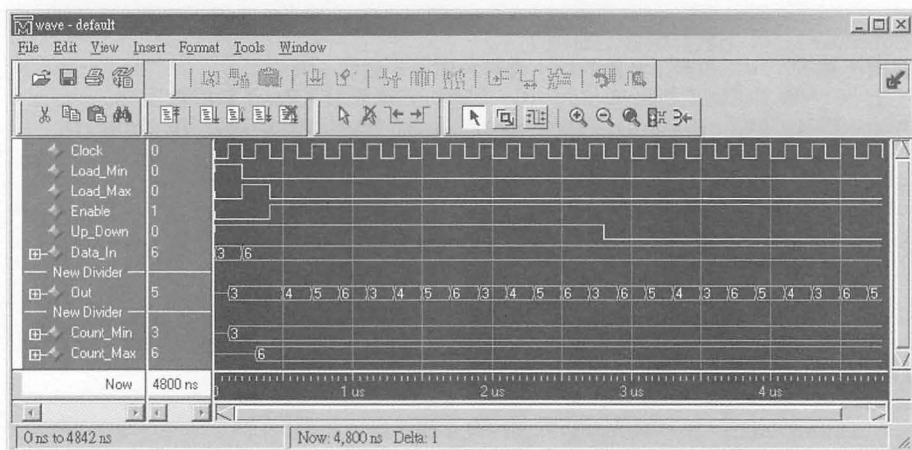
例 7 具有「預設」計數的最小及最大值、「致能輸出」及「上／下數」訊號控制的 8 位元 (0~255) 正緣觸發上數/下數計數器



```
// Counter7.V:上數計數器, 範例 7
// 具有「重置」、「預設」計數的最小及最大值、「致能」及
// 「上 / 下數」訊號控制的 8 位元 (0~255) 正緣觸發上數/下數計數器
module FSM_Cnt1 (Clock, Load_Min, Load_Max, Enable, Up_Down,
                 Data_In, Out);
input          Clock, Load_Min, Load_Max, Enable, Up_Down;
input [7:0]    Data_In;
output [7:0]    Out;
reg [7:0]      Out;
reg [7:0]      Count_Min;
reg [7:0]      Count_Max;

always @ (posedge Clock)
begin
    if (Load_Min) // 載入計數範圍的最小值
    begin
        Out = Data_In; // 預設計數最小值
        Count_Min = Data_In; // 預設計數最小值
    end
    else
    if (Load_Max) // 載入計數範圍的最小值
        Count_Max = Data_In; // 預設計數最大值
    else
    if (Enable) // 致能計數
    begin
        if (Up_Down) // 上數
        begin
            if (Out < Count_Max)
                Out = Out + 1; // 計數值 +1
            else
                Out = Count_Min; // 設為計數最小值
        end
        else // 下數
        begin
            if (Out > Count_Min)
                Out = Out - 1; // 計數值 -1
            else
                Out = Count_Max; // 設為計數最大值
        end
    end
end
endmodule
```

在底下的模擬中：我們讓計數的最小值等於 3，計數的最大值等於 6，所以計數器輸出的結果一直介於 3 到 6 之間。



- ▲ Counter7.V：具有「重置」、「預設」計數的最小及最大值、「致能輸出」及「上/下數」訊號控制的 8 位元（0~255）正緣觸發上數／下數計數器合成後，電路的模擬結果



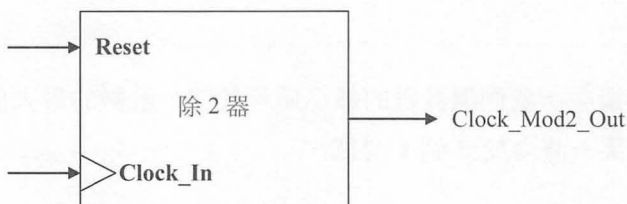
9.4 除頻電路（Frequency Divider）

除頻電路主要是由計數電路衍生而來的。

除頻電路，依其除數值，可分為下列二種：

- 固定除以某個大小的「除 N 器」（N 為一個正整數值）。
- 可以由使用者自訂 N 的大小的「可程式的除頻器」。

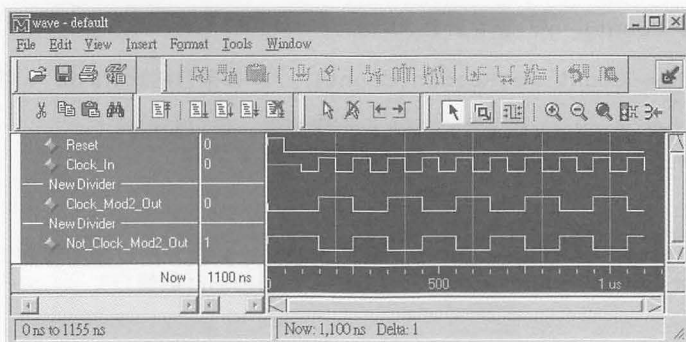
例 1 「除 2 器」（N = 2），將輸入時脈訊號頻率除以 2 之後，再輸出的邏輯符號



```
// FreqMod2.V: 除 2 器
// 將輸入時脈訊號頻率除以 2 以後再輸出
module FreqMod2 (Reset, Clock_In, Clock_Mod2_Out);
input  Reset, Clock_In;
output Clock_Mod2_Out;
reg    Clock_Mod2_Out;
wire   Not_Clock_Mod2_Out;

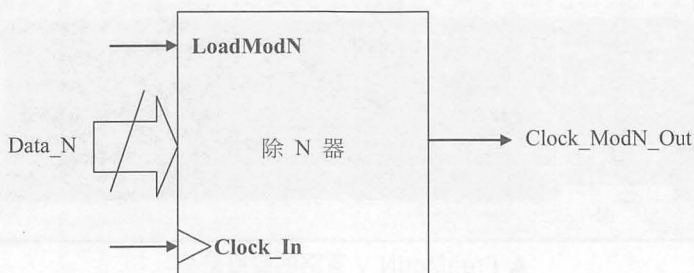
    assign Not_Clock_Mod2_Out = !Clock_Mod2_Out;

    always @(posedge Reset or posedge Clock_In)
    begin
        if (Reset)
            Clock_Mod2_Out = 0;
        else
            Clock_Mod2_Out = Not_Clock_Mod2_Out;
        end
    endmodule
```



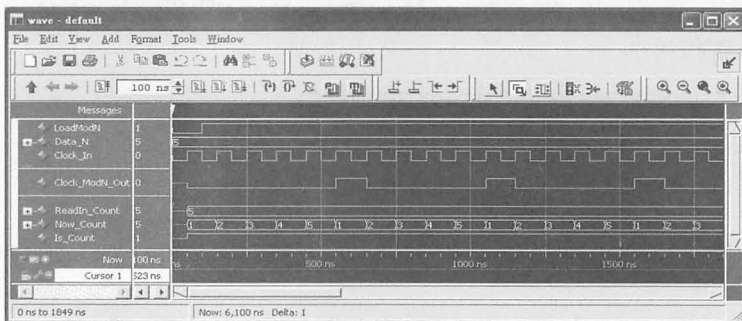
▲ FreqMod2.V 合成後，電路的模擬結果

例 2 「除 N 器」，可程式的除頻器（將輸入時脈訊號頻率除以 N 之後再輸出）的邏輯符號



```
// FreqModN.V: 除 N 器
// 可程式的除頻器 (將輸入時脈訊號頻率除以 N 以後再輸出)
module FreqModN (LoadModN, Data_N, Clock_In, Clock_ModN_Out);
input      LoadModN;
input      [3:0] Data_N;
input      Clock_In;
output     Clock_ModN_Out;
reg        Clock_ModN_Out;
reg        [3:0] ReadIn_Count, Now_Count;
reg        Is_Count;

always @(posedge Clock_In) // 正緣脈波訊號
begin
    if (!LoadModN) // 同步負單位載入
    begin
        Clock_ModN_Out = 0; // 才剛要開始計數, 所以輸出為 0
        Now_Count = 1; // 目前計數到 1
        ReadIn_Count = Data_N; // 讀入欲計數的次數 (除頻數)
        Is_Count = 1; // 設定正在計數旗標為 1
    end
    else
    begin
        if (Is_Count) // 正在計數?
        begin
            if (Now_Count == ReadIn_Count) // 如果目前計數的值 = 除頻數
            begin
                Clock_ModN_Out = 1; // 已到除頻值, 所以輸出為 1
                Now_Count = 1; // 新設定目前計數值為 1
            end
            else
            begin
                Clock_ModN_Out = 0; // 尚未計數到除頻數, 所以輸出為 0
                Now_Count = Now_Count + 1; // 目前的計數值 +1
            end
        end
    end
end
endmodule
```



▲ FreqModN.V 電路的模擬結果

本章練習**Question 1 :**

請詳細比較並說明 D 型門鎖 (Latch) 與 D 型正反器 (D type Flip-Flop) 的差異以及使用時機。

Question 2 :

請說明：串進串出 (Serial In Serial Out) 移位暫存器、串進並出 (Serial In Parallel Out) 移位暫存器、並進串出 (Parallel In Serial Out) 移位暫存器以及並進並出 (Parallel In Parallel Out) 移位暫存器的使用時機。

Question 3 :

請說明：上數計數器 (Up Counter)、下數計數器 (Down Counter) 以及上數/下數計數器 (Up/Down Counter) 的使用時機。

PS Google 可以幫您找到解答。

